Heap

A few points:

There are many words in Computer Science which have different meanings depending on the context. Already we have observed that thread in Operating Systems stands for a unit of execution whereas threaded tree makes the NULL structural pointers point to its successor or predecessor making iterative functions possible for a particular order of traversal.

Similarly, the word heap has 2 meanings. Heap in memory management refers to a part of the memory supporting allocation and deallocation of memory dynamically.

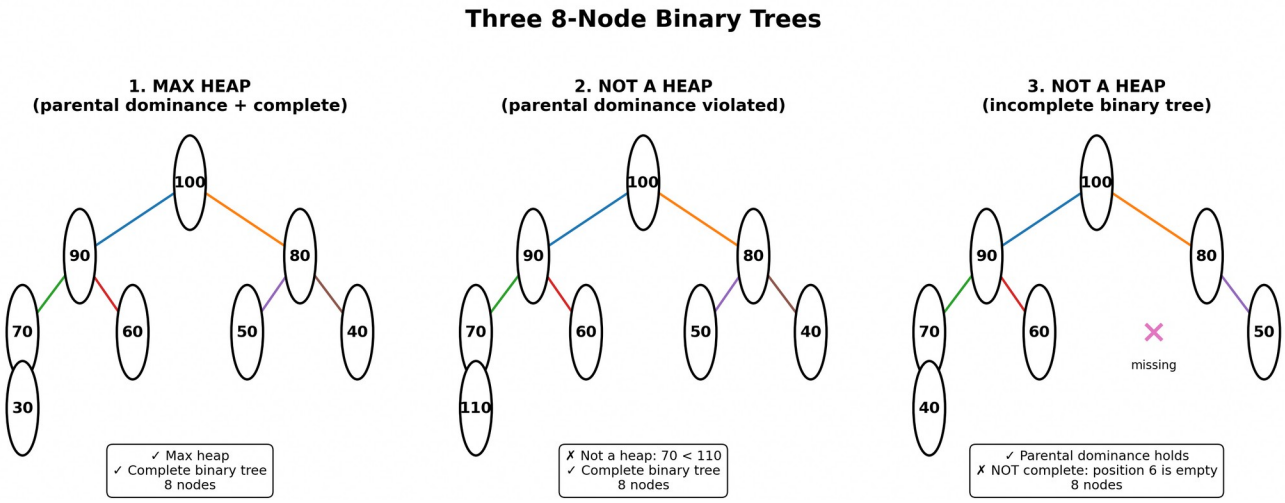
In data structure, a heap refers to a special type of binary tree. It is not a search tree.
There are two kinds of heaps – called maxheap and minheap.

In our discussion, we will discuss the maxheap. The interface and the implementation of minheap will be similar – we have to change the comparison operator from “greater than” to “less than”.

A heap(maxheap) is a tree with two special properties.

1. Complete binary tree:
   The nodes are filled up at each level.
   At the bottom most level, leftmost nodes are filled up.
2. Parental dominance:
   The key at the root is greater than the keys of its subtrees recursively.

Three diagrams in the next page illustrate the properties of the heap tree.
The first one is a heap. The root has the highest value and keys at the roots of the subtree are less than that of the root. There is no ordering of keys at the left and the right subtrees. This must be true recursively. Evidently, leaves are always heaps. Also, observe that the first 3 levels are totally filled up. In the last level, the leftmost nodes are filled up.



The second example is not a heap as the child of node with the key 70 has the key 110. It does not satisfy parental dominance. So, this is not a heap.

In the third example, the second level(counting from 0) is not completely filled. The left subtree of node with key 80 is missing. So this is not a heap. This tree has 7 nodes.

Implementation of a heap tree:

We always implement a heap tree using an array. As the tree is a complete tree, it provides an easy of mapping to an array and also provides simple ways by which we can make out whether the tree is full or empty, insert, traverse and so on.

We have two ways of mapping a heap to an array.

1. We may start with the root being mapped to 0th element of the array.

   In such a case, we use the following relationships to relate the root, the left subtree and the right subtree.

   Leftchild(i) = 2i + 1

   Rightchild(i) = 2i + 2

   Parent(i) = floor((i-1)/2)

2. We may start with the root being mapped to 1st element of the array.

   In such a case, we use the following relationships to relate the root, the left subtree and the right subtree.

   Leftchild(i) = 2i

   Rightchild(i) = 2i + 1

   Parent(i) = floor(i/2)

In our discussion, we will use the second implantation.

Advantages of heap implementation:

- No pointers
- No extra storage
- Easy way to determine empty or full
- Simple way to traverse either as an array or as a tree
- Does not waste memory as the tree is a complete binary tree

Uses of Heap :

1. Heapsort : There is way of sorting using the concept of heap. This is theoretically one of the most efficient algorithm. It is as efficient as any other algorithm for sorting based on comparison. But, this algorithm is not cache friendly. So, no library of any programming language implements this algorithm.
2. Finding maximum (or minimum) repeatedly:

   In many applications, we may have insert and delete based on the order of the keys. The classical example is priority queue. We enqueue a task and then dequeue a task based on priority. In such cases, both these operations are equally efficient and every one of them requires less number of operations compared to the number of elements in the tree. In fact, the number of operations is proportional to the logarithm to base 2 of the number of elements in the tree.

Example 1 :

Build a heap:

Input : array having n elements

Output : array representing a maxheap of n elements

Structure of heap:

This heap has an array of keys and has the number of elements in use.

```
struct heap
{
    int key_[MAXSIZE];
    int n_;
};
typedef struct heap  heap_t;
```

Interface of heap :

```
void init(heap_t *ptr_heap);

void deinit(heap_t *ptr_heap);

void disp(heap_t *ptr_heap);


void build_heap(heap_t *ptr_heap, int a[], int n);
```

Implementation:

- **Initialize and deinitialize:**

  There is no dynamic allocation. The number of elements in the tree is made 0

  ```
  ptr_heap->n_ = 0;
  ```

- **Display**

  This function displays the element of the tree as an array varying index from 1 to n.

Build a heap:

This takes an array as argument and copies the array to the heap data structure and converts it to a heap.

There are 2 parts in the algorithms.

The first part copies the given array into the array in the heap structure.

The second part tries to convert every subtree of this array representation to a heap.

All leaves are heaps. So, the conversions start from the middle of the array skipping all leaves and moves towards the root.

The function heapify converts the tree rooted at position i.

We will discuss heapify in the next section.

```
void build_heap(heap_t *ptr_heap, int a[], int n)
{
    ptr_heap->n_ = n;
    for(int i = 1; i <= n; ++i)
    {
            ptr_heap->key_[i] = a[i];
    }
    for(int i = n / 2; i >= 1; --i)
    {
            heapify(ptr_heap->key_, ptr_heap->n_, i);
            disp(ptr_heap);
    }
}
```

Heapify:

Given that the left subtrees and the right subtrees of a node are already heap, we want to position the root element properly in the subtrees so that the tree rooted at the given root becomes a heap.

The position of the root is i.

Its left child is at position 2i. This is stored in the variable j.

The value at the root i is copied so that the ith location becomes a hold – can be modified.

We enter a loop when j is still a valid index(j <= n) and the root(ith) element is not greater than the keys at the roots of its subtrees(not a heap yet).

If j+1 key exists and is greater than the jth key, we move to the right by incrementing j.

If jth key is less than the value of the root, out search is over.

Otherwise, move the jth subtree by shifting the hold and changing i and j.

When we exit the loop, we copy the val to the ith location.

```
static void heapify(int a[], int n, int pos)
{
    int i = pos;
    int j = 2 * i;
    int val = a[i];
    int is_heap = 0;
    while(j <= n && ! is_heap)
    {
        if(j + 1 <= n && a[j+1] > a[j])
            ++j;
        if(a[j] > val)
        {
            a[i] = a[j];
            i = j;
            j = 2 * i;
        }
        else
        {
            is_heap = 1;
        }

    }
    a[i] = val;
}
```

Example 2:

Build a menu driven program to support insertion and deletion:

In this example, a few functions are added.

Interface:

```
void insert(heap_t *ptr_heap, int key);
int delete_max(heap_t *ptr_heap);
int peek_max(heap_t *ptr_heap);
```

insert a key:

In insert, we always add the element at the end of the array and increase the heap size by 1. Then we propagate the element up towards the root to make the tree a heap.

```
void insert(heap_t *ptr_heap, int key)
{
    // add at the end
    // heapify upwards
    int pos = ++ptr_heap->n_;
    if(pos > MAXSIZE)
    {
            printf("heap full\n"); exit(1);
    }
    ptr_heap->key_[pos] = key;
    while(pos > 1 && ptr_heap->key_[pos] > ptr_heap->key_[pos/2])
    {
            swap(&ptr_heap->key_[pos], &ptr_heap->key_[pos/2]);
            pos = pos / 2;
    }
}
```

The key is added at the end. The number of elements in the tree is incremented.

If the key added is greater than that of its parent, we interchange the two and repeat the operation until the above condition becomes false or we reach the root.

In heapify routine, we propagated a hole. In this we have used the concept of swapping instead.

Delete key:

In delete, we always delete the max element which is at the root. We overwrite this root element with the last element of the array, decrease the tree size by 1 and then heapfiy about the root.

```
int delete_max(heap_t *ptr_heap)
{
    if(ptr_heap->n_ == 0)
    {
            printf("heap empty\n"); exit(1);
    }
    int res = ptr_heap->key_[1];
    ptr_heap->key_[1] = ptr_heap->key_[ptr_heap->n_];

    heapify(ptr_heap->key_, --ptr_heap->n_, 1);
    return res;
}
```

Peek max :

This will return the value at the root – the key at index 1.

```
int peek_max(heap_t *ptr_heap)
{
    return ptr_heap->key_[1];
}
```

Observe that in delete, we call heapify, which discards one half the tree each time by doubling the value of j.

Observe that in insert, we keep moving towards the parent and do not move towards the sibling by halving the index each time.

So, in both the cases, the number of operations we do is proportional to the log to base 2 of number of elements in the tree .

Example 3:

Priority queue using heap:

Structure of priority queue:

```
struct node
{
    int key_;
    struct node* next_;
};
typedef struct node node_t;

struct queue
{
    heap_t q_;
};
typedef struct queue queue_t;
```

The priority queue has composition of heap in it.

Interface of priority queue:
All these functions will in turn delegate the functionality to those of the heap.

```
void init_queue(queue_t *ptr_queue);
void deinit_queue(queue_t *ptr_queue);
void enqueue(queue_t *ptr_queue, int key);
int dequeue(queue_t *ptr_queue);
int is_empty(queue_t *ptr_queue);
int is_full(queue_t *ptr_queue);
```

Implementation of priority queue:

```
void init_queue(queue_t *ptr_queue)
{
    init(&ptr_queue->q_);
}

void deinit_queue(queue_t *ptr_queue)
{
    deinit(&ptr_queue->q_);
}



void enqueue(queue_t *ptr_queue, int key)
{
    insert(&ptr_queue->q_, key);

}
int dequeue(queue_t *ptr_queue)
{
    return delete_max(&ptr_queue->q_);
}
```

Here, we end the discussion of heap.

Please note that the sorting based on heap – heapsort is never used in any library.

Also, whenever insertion and deletion are repeatedly executed and are based on a key, heap the most preferred data structure.